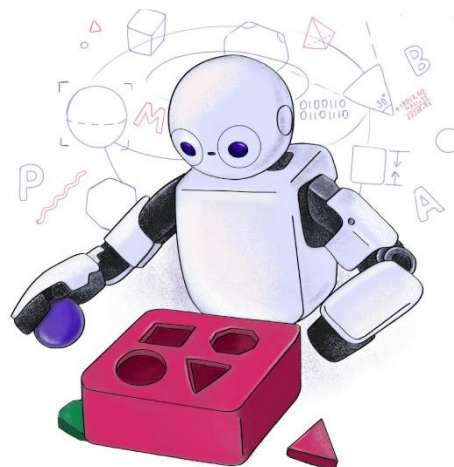
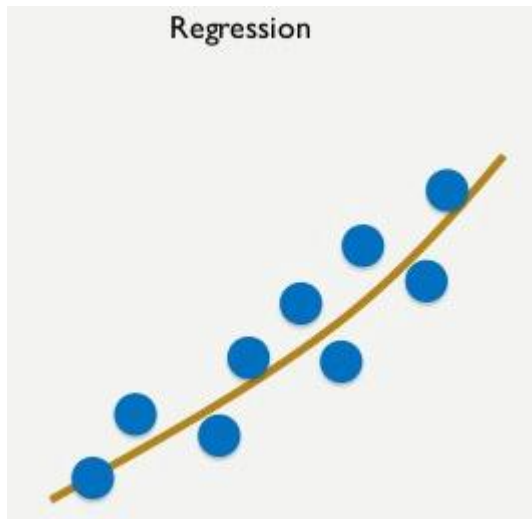


C24 – Inteligência Artificial: *Classificação*



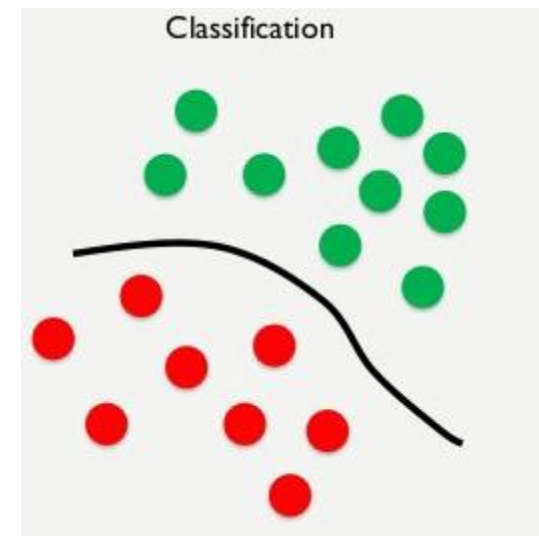
Classificação

- Tarefa (ou problema) de **aprendizado supervisionado**.
 - As saídas esperadas (rótulos) são conhecidas.
- Envolve encontrar uma função, $f(x)$, que *mapeie* os atributos de entrada em um *conjunto finito de valores discretos*, ou seja, em classes.



$f(x)$ captura o *comportamento geral* dos dados.

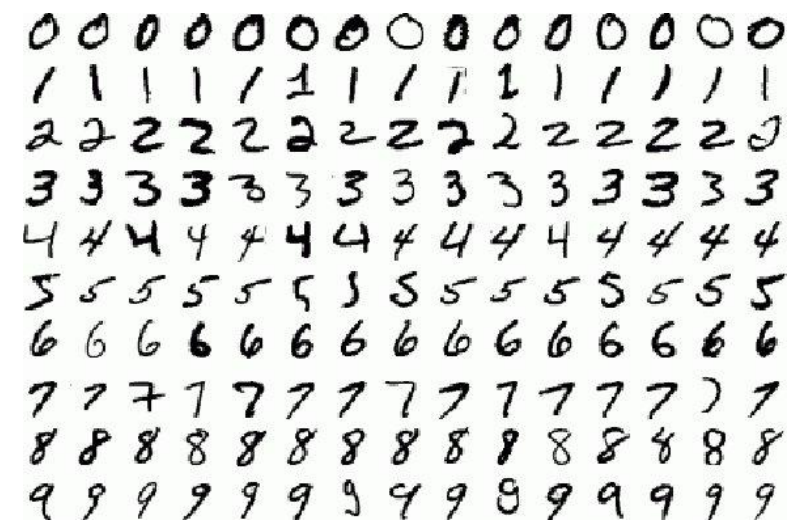
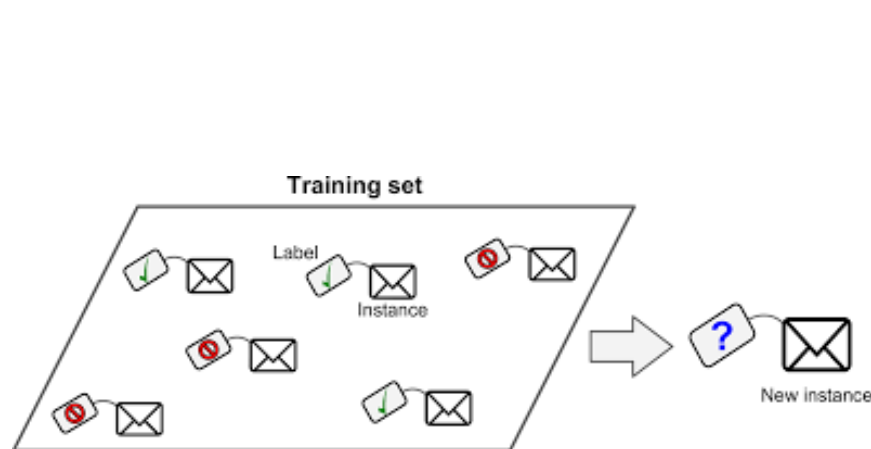
$f(x)$ *aproxima* o comportamento dos dados.



$f(x)$ forma uma *fronteira de decisão*.

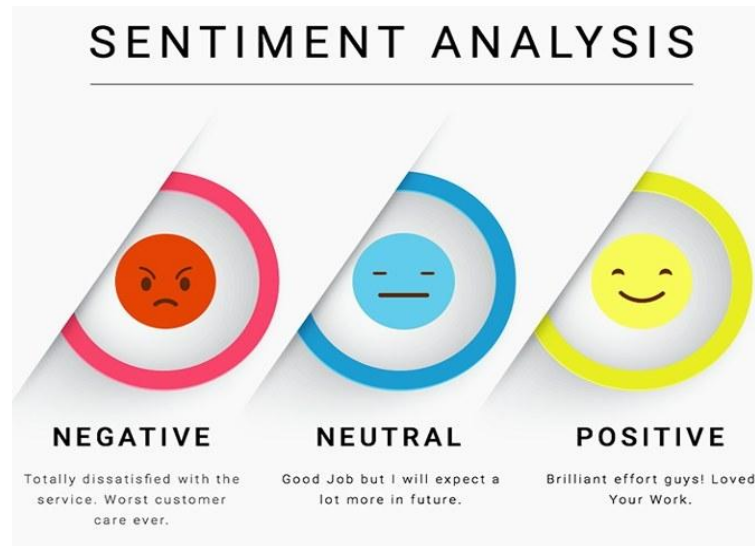
$f(x)$ *classifica* os dados.

Tarefas de classificação



- Classificação de emails entre *spam* e *ham* (legítimo).
- Classificação de objetos em imagens.
- Reconhecimento de caracteres.

Tarefas de classificação



- Classificação de texto (e.g., notícias).
- Classificação de sentimentos.
- Classificação do doenças (e.g., pulmonares).

**WHEN YOU TEST YOUR FIRST
IMAGE CLASSIFIER MODEL**

MONKEY



ZEBRA



TIGER



FROG



Tipos de classificadores

- Organizamos os tipos de classificadores segundo dois critérios principais:
 - Número total de classes possíveis, Q , e
 - Quantidade de classes atribuídas a cada amostra.
- Dependendo do número de classes possíveis, Q , podemos separar os classificadores em dois tipos:
 - Binários, onde $Q = 2$.
 - Existem apenas **duas classes possíveis**
 - Multiclasses, onde $Q > 2$.
 - Existem **três ou mais classes**
- Em ambos os casos, cada amostra pertence a **uma única classe**.

Tipos de classificadores

- Podemos também separar os classificadores dependendo de quantas classes podem ser atribuídas a uma única instância:
 - Classificação *single-label*
 - Cada amostra recebe apenas uma classe.
 - Inclui: binária e multiclasse
 - As classes são mutuamente exclusivas.
 - Classificação *multilabel*
 - Uma amostra pode pertencer a várias classes simultaneamente
 - As classes não são mutuamente exclusivas
 - Exemplos:
 - ✓ Imagem com “cachorro” e “pessoa”,
 - ✓ Artigo com múltiplos tópicos.
 - O modelo prevê um vetor binário indicando quais rótulos estão presentes.

Algoritmos de classificação

- Existe uma grande quantidade de algoritmos de classificação, mas focaremos nos seguintes
 - Regressor logístico
 - Classificador binário e *single-label*.
 - Regressor softmax
 - Classificador multiclass e *single-label*.
- Discutiremos adiante também algoritmos que resolvem problemas de regressão e classificação, como
 - Árvores de decisão
 - Classificadores binários, multiclass e *single-label*
 - Redes neurais
 - Classificadores binários, multiclass, *single-label* e *multilabel*

Como representamos os rótulos?

Classificação binária

- O classificador tem **única saída escalar** para indicar a **classe** a que pertence o i -ésimo **vetor de atributos**, $\mathbf{x}(i)$.
- Os rótulos são representados como:

$$y(i) = \begin{cases} 0, & \mathbf{x}(i) \in C_1 \\ 1, & \mathbf{x}(i) \in C_2 \end{cases},$$

onde C_1 e C_2 representam as classe negativa e a positiva, respectivamente.

- Também é possível utilizar $y(i) = -1$ para $\mathbf{x}(i) \in C_1$, ou seja

$$y(i) = \begin{cases} -1, & \mathbf{x}(i) \in C_1 \\ 1, & \mathbf{x}(i) \in C_2 \end{cases}.$$

Classificação multiclasse

- O classificador tem **Q saídas escalares, uma para cada classe**, para indicar a classe a que pertence o i -ésimo vetor de atributos, $\mathbf{x}(i)$.
- Os rótulos são representados como **vetores binários** $\in \mathbb{Z}_+^{Q \times 1}$.
- **Exemplo:** classificação de imagens em 4 **classes mutuamente excludentes**: gato, rato, cachorro e sapo:

$$y(i) = \begin{cases} [1 & 0 & 0 & 0]^T, & \mathbf{x}(i) \in C_{\text{gato}} \\ [0 & 1 & 0 & 0]^T, & \mathbf{x}(i) \in C_{\text{rato}} \\ [0 & 0 & 1 & 0]^T, & \mathbf{x}(i) \in C_{\text{cachorro}} \\ [0 & 0 & 0 & 1]^T, & \mathbf{x}(i) \in C_{\text{sapo}} \end{cases}.$$

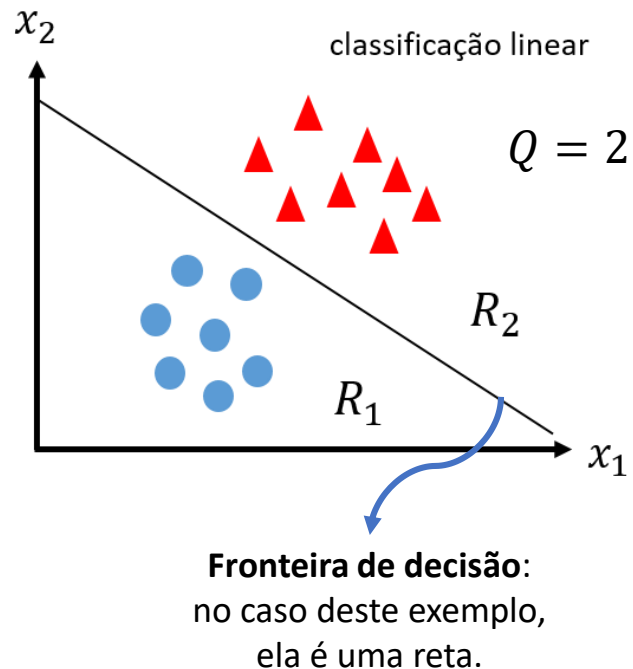
- Essa representação é conhecida como codificação **one-hot**.
- Muito usado em redes neurais e regressão *softmax*.

Classificação *multilabel*

- O classificador tem **Q saídas escalares, uma para cada classe**, porém, cada i -ésimo vetor de atributos, $\mathbf{x}(i)$ pode pertencer a **várias classes simultaneamente**.
- Rótulos também são representados como **vetores binários** $\in \mathbb{Z}_+^{Q \times 1}$.
- **Exemplo**: classificação de imagens em 4 classes **não excludentes**: gato, rato, cachorro e sapo:

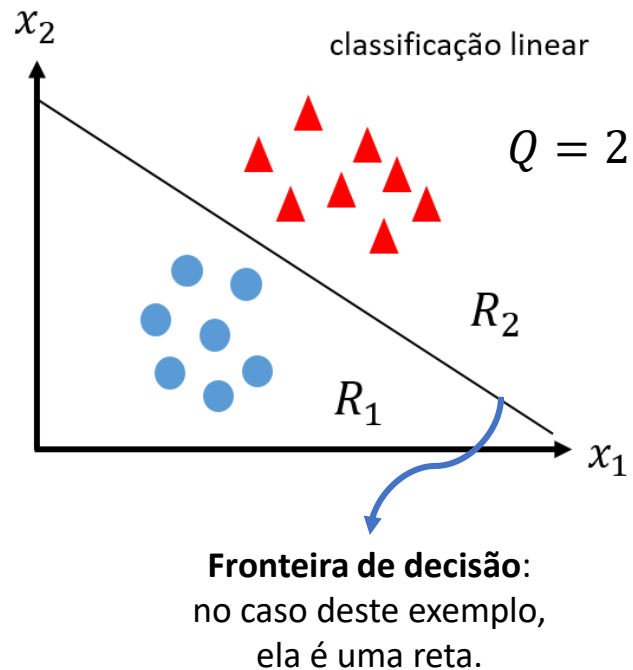
$$y(i) = \begin{cases} [1 & 0 & 0 & 0]^T, & \mathbf{x}(i) \in C_{\text{gato}} \\ [1 & 1 & 0 & 0]^T, & \mathbf{x}(i) \in C_{\text{rato}} \wedge C_{\text{gato}} \\ [1 & 0 & 1 & 1]^T, & \mathbf{x}(i) \in C_{\text{gato}} \wedge C_{\text{cachorro}} \wedge C_{\text{sapo}} \\ [1 & 1 & 1 & 1]^T, & \mathbf{x}(i) \in C_{\text{gato}} \wedge C_{\text{rato}} \wedge C_{\text{cachorro}} \wedge C_{\text{sapo}} \end{cases}.$$

Fronteiras de decisão de um classificador



- Na regressão, usamos **funções hipótese** para **aproximar o comportamento do conjunto de dados**, agora, as usaremos para **separar (i.e., discriminar) grupos de dados em classes**.
- Vejam o **espaço bi-dimensional**, $\mathbb{R}^2$, criado pelos **atributos** x_1 e x_2 , mostrado na figura ao lado.
- Os **pares de atributos** pertencem a duas classes ($Q = 2$):
 - Círculos azuis pertencem à classe C_1 .
 - Triângulos vermelhos pertencem à classe C_2 .

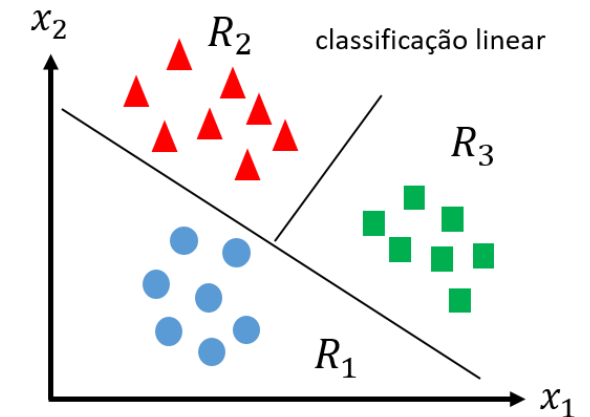
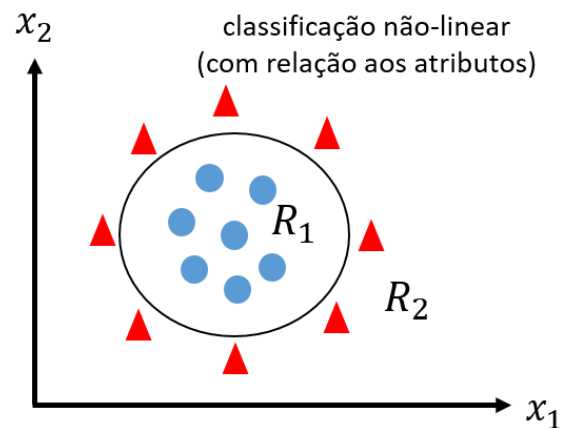
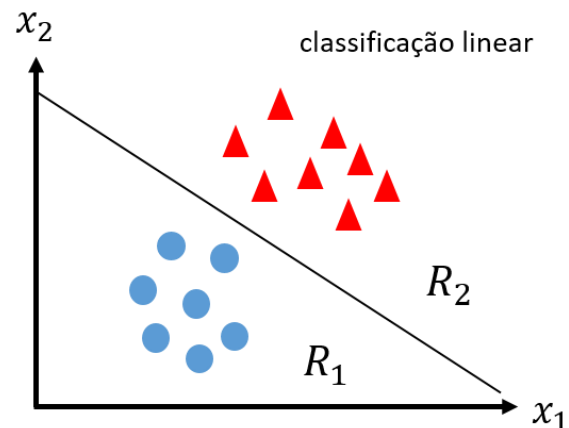
Fronteiras de decisão de um classificador



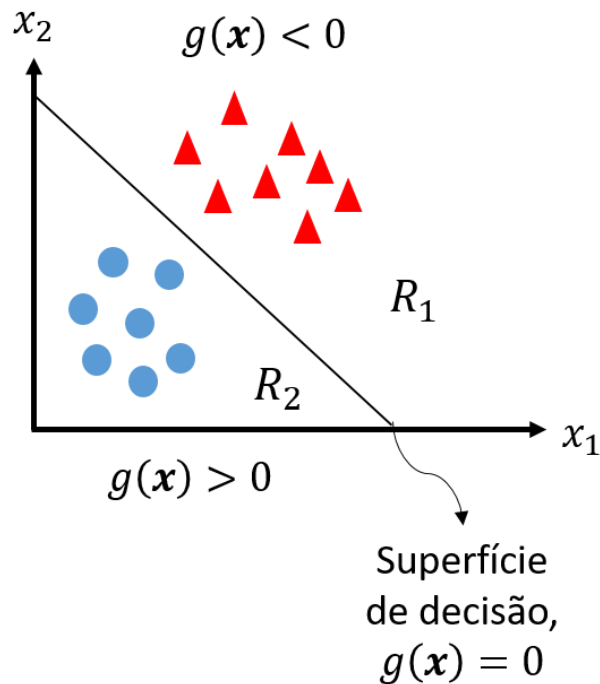
- O objetivo da classificação é encontrar uma função, chamada de **discriminante**, $g(\mathbf{x})$, que separe as classes.
- Chamamos de **fronteira de decisão** onde essa função corta o espaço de atributos.
- No exemplo, a função discriminante **divide** o espaço em **duas regiões de decisão**, R_1 e R_2 , onde **cada região corresponde a uma classe**.
- No exemplo, como $Q = 2$, temos apenas uma fronteira de decisão.

Fronteiras de decisão de um classificador

- As figuras abaixo mostram **funções discriminantes** em problemas de classificação **binária** e **multiclasses**.
- Elas podem ser **lineares** (e.g., retas e planos) ou **não-lineares** (e.g., círculos e elipses).
- Elas formam **superfícies de separação** no espaço de atributos que podem ter várias dimensões (2D, 3D, etc.).
- Em geral, usamos polinômios como **funções discriminantes**.



Funções discriminantes

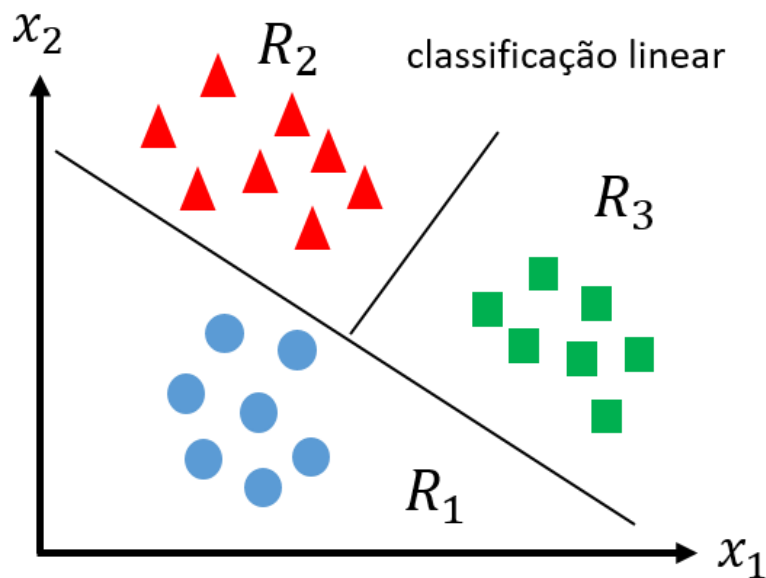


- No caso **binário**, nosso **objetivo** é encontrar o **grau e os pesos da função discriminante (polinomial)** de tal forma que a classe atribuída a $\mathbf{x}(i)$ seja:

$$C_q = \begin{cases} q = 1, & g(\mathbf{x}(i)) < 0 & \text{(Amostra acima da função)} \\ q = 2, & g(\mathbf{x}(i)) > 0 & \text{(Amostra abaixo da função)} \\ \text{Indeterminação,} & g(\mathbf{x}(i)) = 0 & \text{(Amostra sobre a função)} \end{cases}$$

- Em caso de **indeterminação**, podemos
 - atribuir arbitrariamente a uma das duas classes,
 - escolher a classe que possui mais amostras ou a mais frequente.
- Quanto mais distante a amostra estiver da fronteira de decisão, maior é o valor absoluto de $g(\mathbf{x})$ e vice versa.

Funções discriminantes



- No caso da classificação **multiclasses**, o **objetivo é encontrar os graus e os pesos das funções discriminantes (polinomiais)** de tal forma que a classe atribuída a um exemplo de entrada, $\mathbf{x}(i)$, seja:

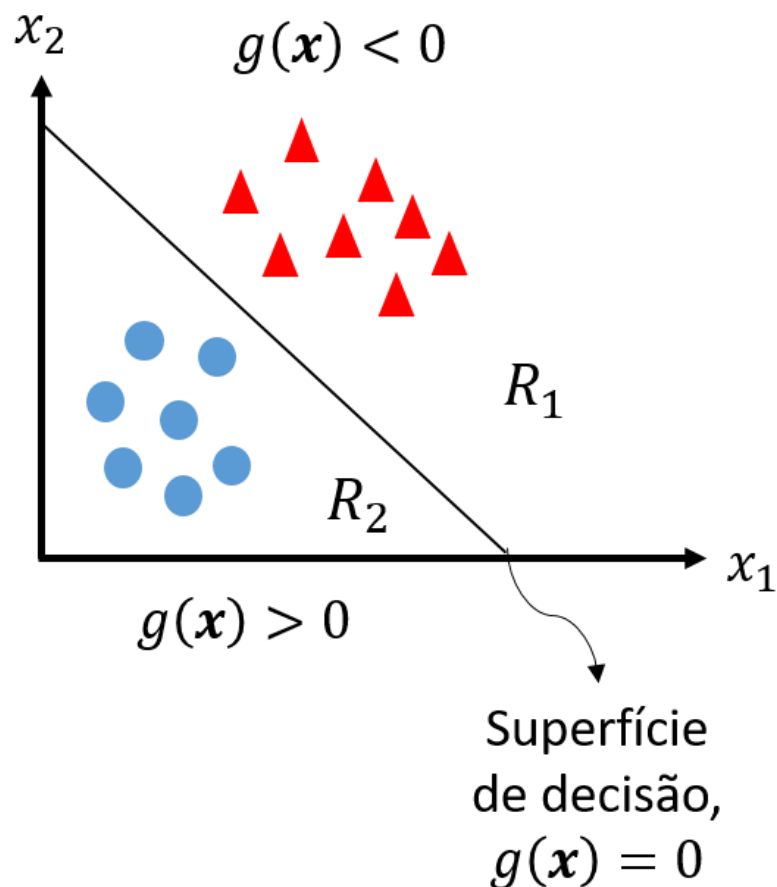
$$C_q = \arg. \max_{q \in \{1, \dots, Q\}} g_q(\mathbf{x}(i)),$$

onde $g_q(.)$ é a q -ésima função discriminante.

*Como encontramos o grau e pesos
das funções discriminantes?*

Da mesma forma que com a regressão, usando o **gradiente descendente** e **validação cruzada**, como o *k-fold*, por exemplo.

Como mapeamos $g(\mathbf{x}(i))$ nas classes?

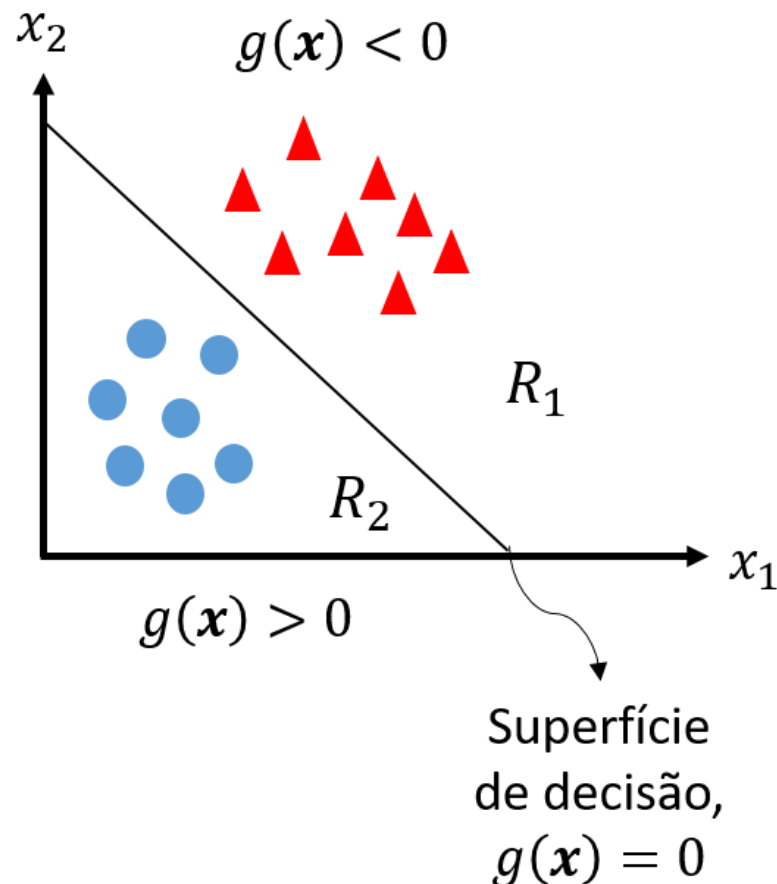


- Vocês perceberam que as saídas das funções discriminantes podem assumir infinitos valores?

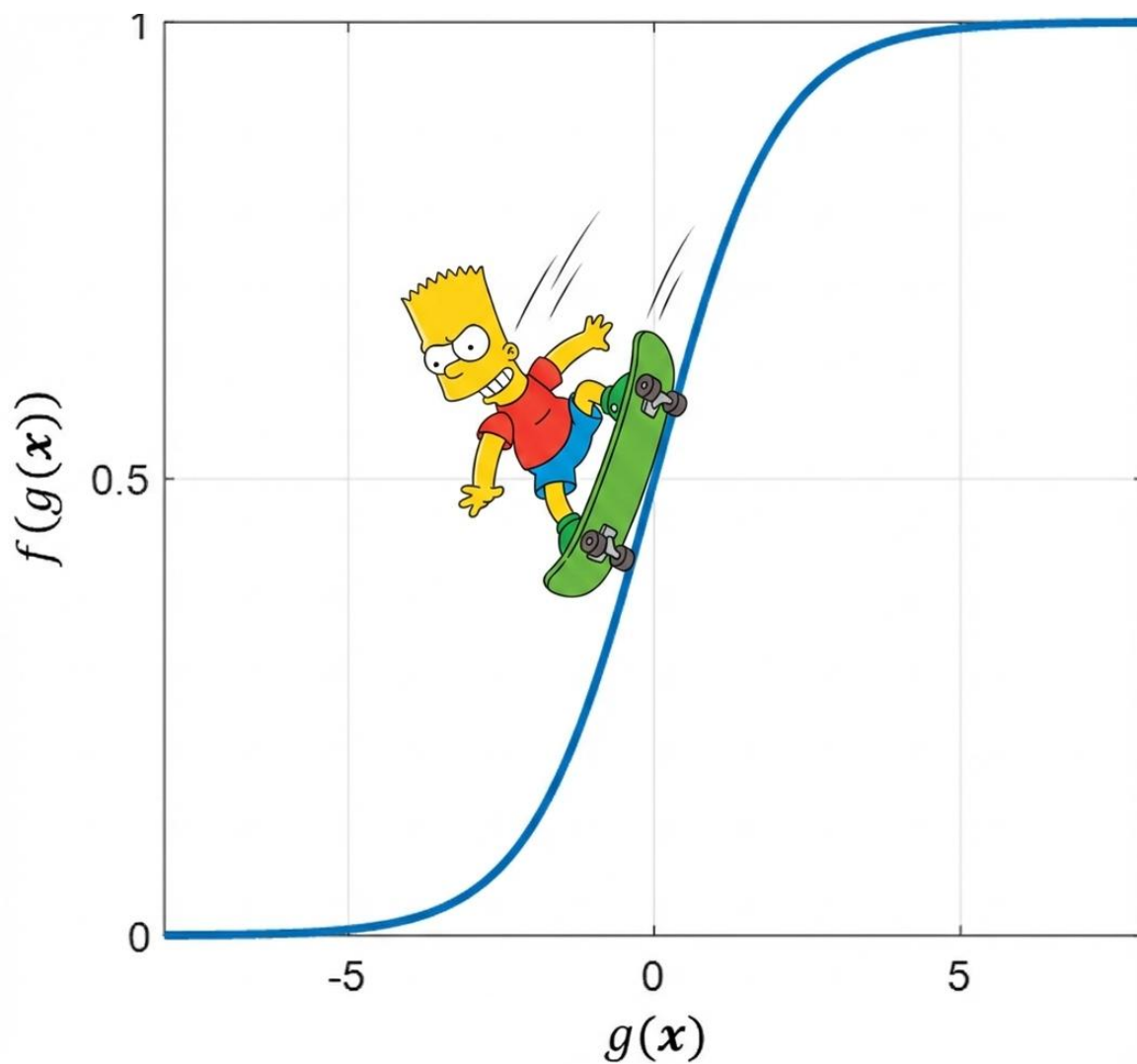
$$g(\mathbf{x}(i)) \in [-\infty, +\infty]$$

- Porém, estamos lidando com problemas de classificação.
 - O objetivo é atribuir uma classe ao vetor de atributos, $\mathbf{x}$.
- Para calcular o erro entre os rótulos, y , e a saída do modelo, $\hat{y}$, ela deve estar no intervalo $[0, 1]$.
- Como mapeamos $g(\mathbf{x}(i))$ nesse intervalo?

Função de limiar de decisão ou de ativação

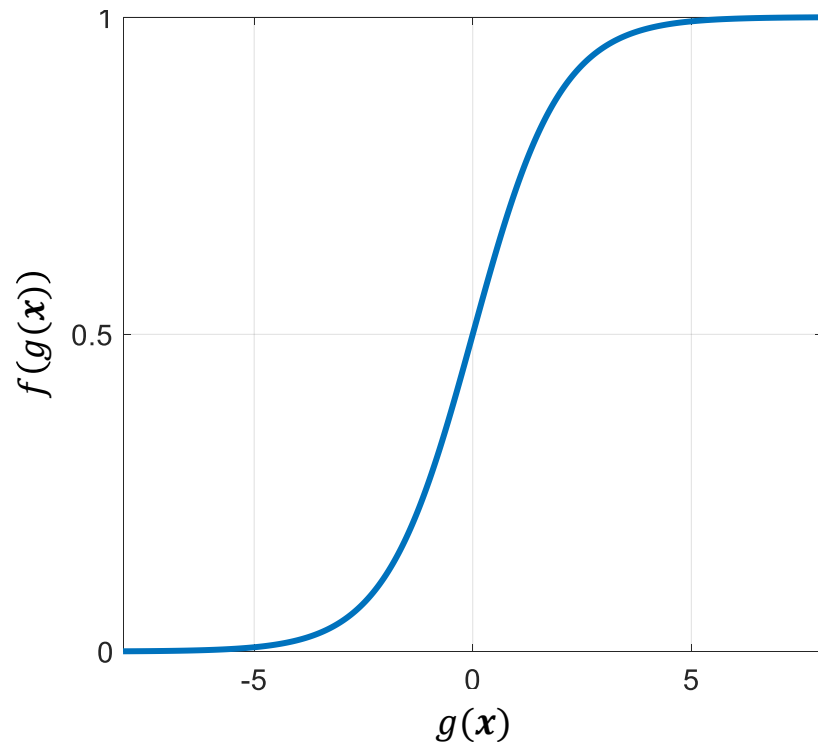


- Isso pode ser feito através de uma função, $f(\cdot)$, que mapeie $g(\mathbf{x}(i))$ no intervalo $[0, 1]$.
- Além disso, para que possamos treinar modelos de classificação, essa função deve ser **contínua** e **diferenciável**.
- No contexto da classificação, ela é chamada de função de limiar de decisão ou de ativação.
- Qual função podemos usar?



Uma função que apresenta tais características é a **função logística**, também conhecida como **sigmoide**.

Classificação com função de limiar logístico



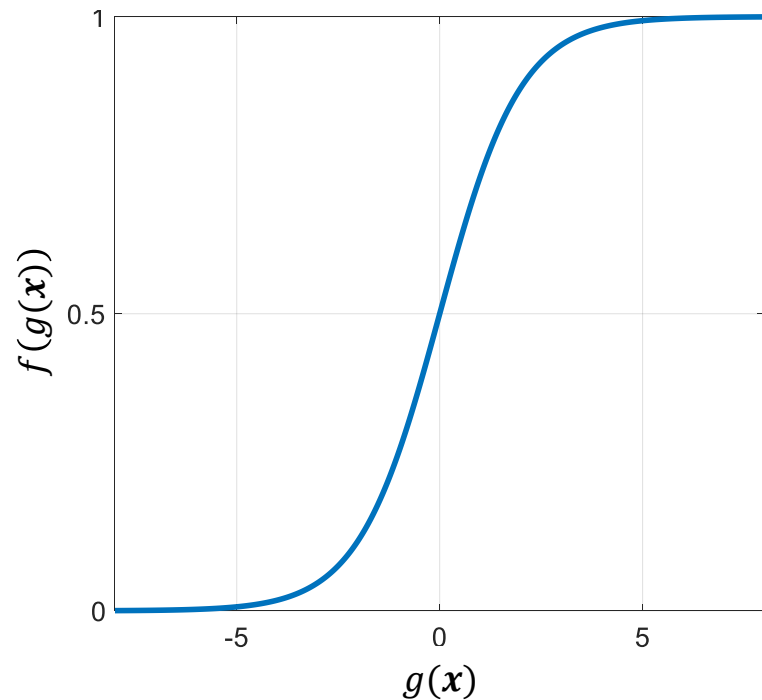
- A **função logística** é definida como

$$\text{Logistic}(z) = \frac{1}{1+e^{-z}} \in [0, 1].$$

- A utilizando como **função de ativação**, temos a seguinte **função hipótese de classificação**

$$h_a(\mathbf{x}) = f(g(\mathbf{x})) = \frac{1}{1+e^{-g(\mathbf{x})}} \in [0, 1].$$

Classificação com função de limiar logístico



- A saída de $h_a(\mathbf{x}(i))$ pode ser interpretada como a probabilidade da classe positiva, C_2 , dado os vetores $\mathbf{x}(i)$ e $\mathbf{a}$, i.e., a probabilidade condicional de C_2

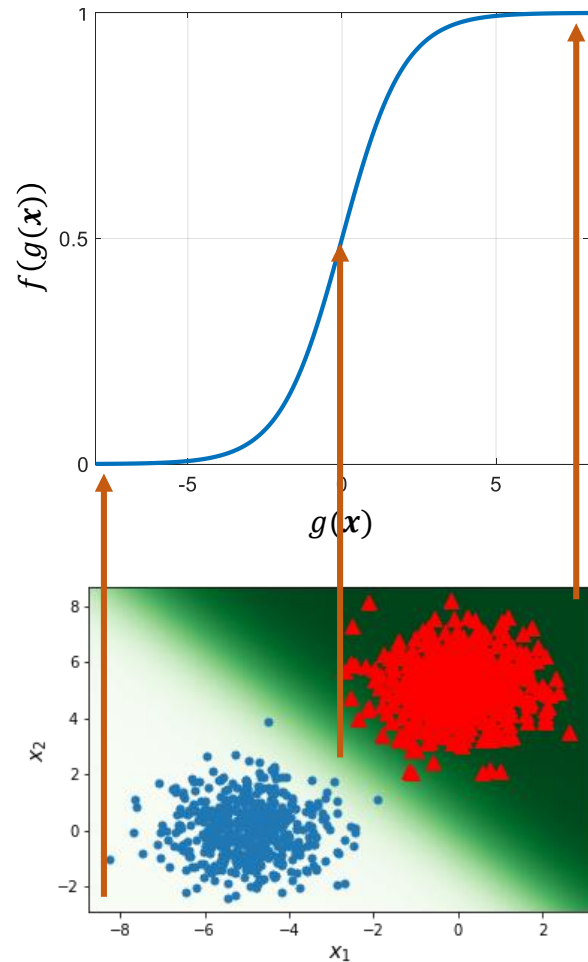
$$h_a(\mathbf{x}) = P(C_2 \mid \mathbf{x}; \mathbf{a}).$$

- Consequentemente, o complemento de $h_a(\mathbf{x})$

$$(1 - h_a(\mathbf{x})) = P(C_1 \mid \mathbf{x}; \mathbf{a}),$$

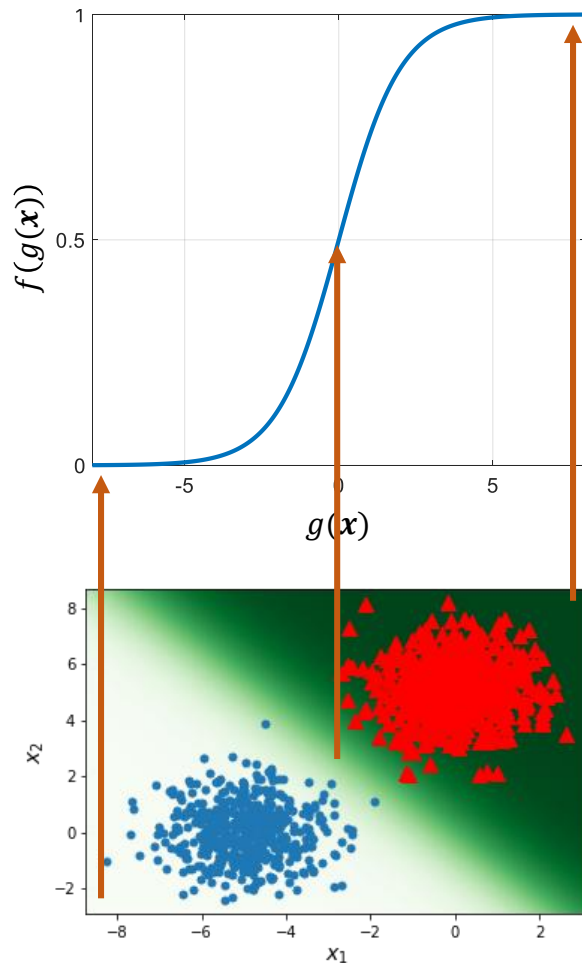
dá a probabilidade condicional da classe negativa, C_1 .

Classificação com função de limiar logístico



- $h_a(\mathbf{x})$ se aproxima de 0 ou 1 conforme o exemplo se distancia da **fronteira de decisão**.
 - Quanto mais distante a amostra estiver da fronteira, maior é a probabilidade dela pertencer à classe daquela região.
- $h_a(\mathbf{x})$ prediz uma probabilidade de 0.5 para amostras posicionadas exatamente sobre a **fronteira de decisão**.
 - A **fronteira de decisão** é definida por $g(\mathbf{x})$ e passa pelos pontos onde $g(\mathbf{x}) = 0$.

Regressão logística



- Esse modelo que estima a probabilidade de uma dada amostra de entrada pertencer à classe positiva, **não é um classificador no sentido estrito**.
- Ele é na verdade um **regressor** e é chamado de **regressor logístico**.
- É um regressor pois sua saída pode **assumir infinitos valores** no intervalo entre 0 e 1.



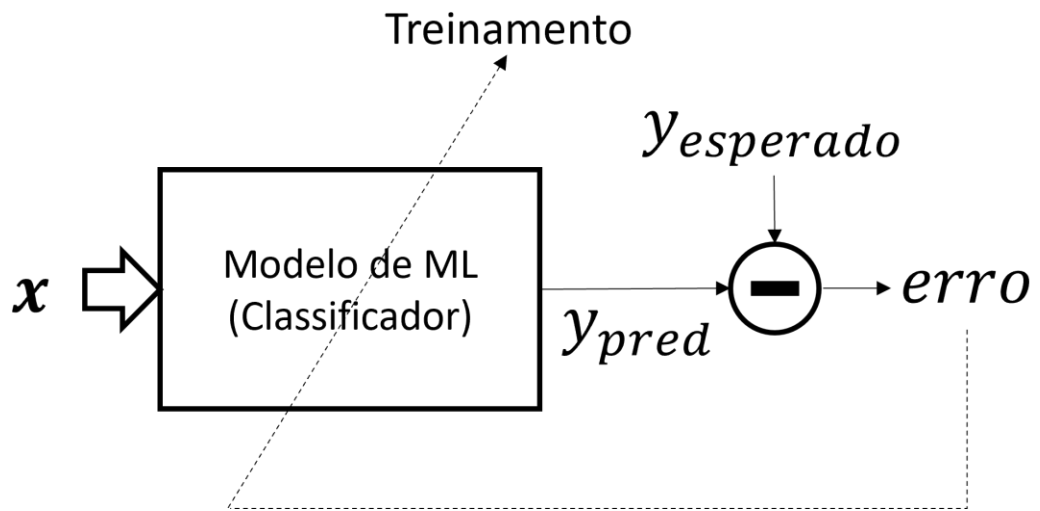
O regressor logístico é usado para **classificação binária**, mas para isso, precisamos **quantizar sua saída**.

Regressão logística

- Se quantiza a saída da **função hipótese**, $h_a(\mathbf{x})$, nos valores 0 ou 1 estabelecendo-se um **limiar de decisão**.
- Em geral, o **limiar de decisão** é feito igual a 0.5.
- A saída **quantizada** do **regressor logístico** é dada por

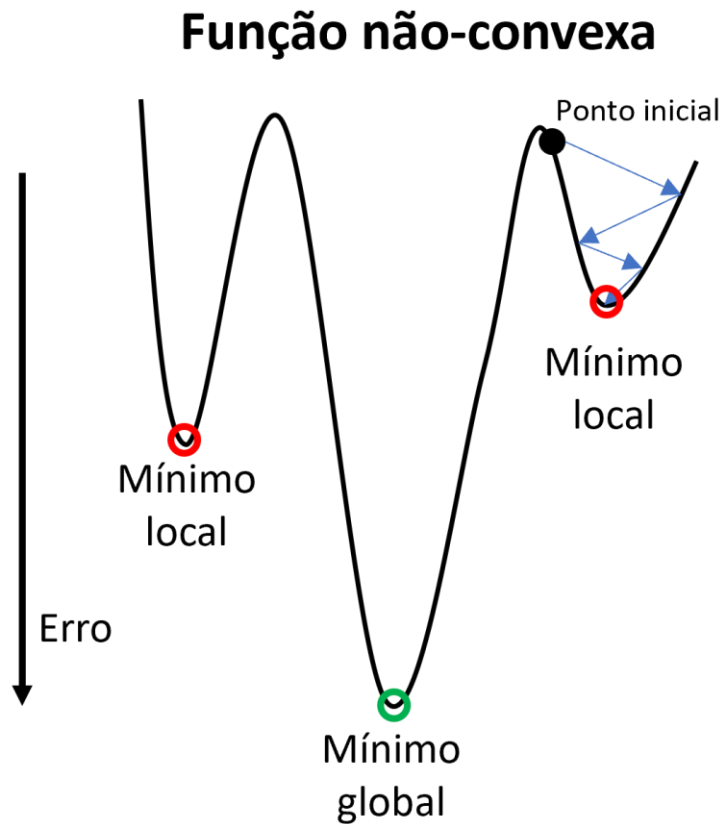
$$\hat{y} = \begin{cases} 0 \text{ (classe } C_1 \text{ – Negativa), se } h_a(\mathbf{x}) < 0.5 \\ 1 \text{ (classe } C_2 \text{ – Positiva), se } h_a(\mathbf{x}) \geq 0.5 \end{cases}.$$

Função de erro



- Com tudo definido, nosso objetivo será, assim como na regressão linear, dado uma função discriminante, encontrar **seus pesos** de forma que o **erro de classificação seja minimizado**.
- Agora precisamos de uma função de erro para **treinarmos um regressor logístico**.
- Porém, usar a função do **erro quadrático médio não é uma boa escolha** no caso da **regressão logística e classificadores em geral** como veremos a seguir.

Função de erro

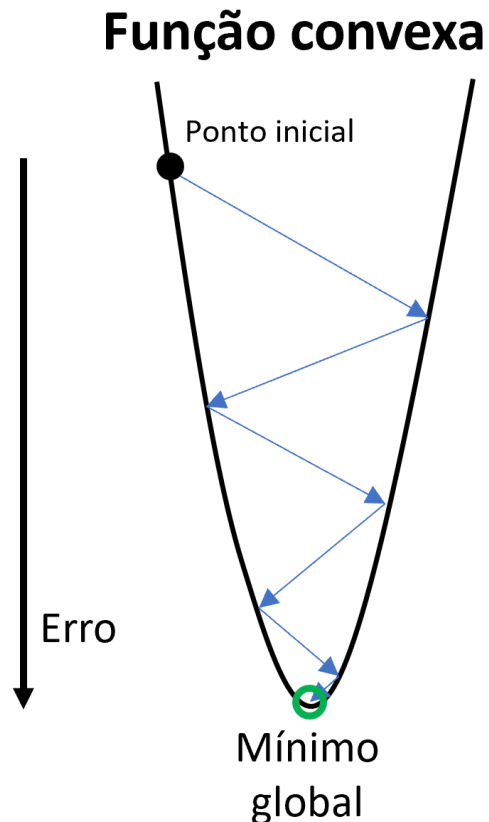


- Se adotarmos o EQM como a **função de erro**, $J_e(\mathbf{a})$, temos

$$J_e(\mathbf{a}) = \frac{1}{N} \sum_{i=1}^N (y(i) - f(g(\mathbf{x}(i))))^2 .$$

- Porém, como $f(\cdot)$ é uma **função não-linear**, $J_e(\mathbf{a})$ **não será**, consequentemente, **uma função convexa**, de forma que a **superfície de erro** poderá apresentar **vários mínimos locais e outras irregularidades** que vão dificultar o aprendizado do modelo.
 - Por exemplo, o algoritmo do GD pode ficar preso em um mínimo local).

Função de erro



- Precisamos usar uma **função de erro** que tenha uma **superfície de erro convexa**.
- Uma função que no caso da regressão logística é **convexa** é a função da **entropia cruzada**
$$J_e(\mathbf{a}) = -\frac{1}{N} \sum_{i=0}^{N-1} y(i) \log(h_{\mathbf{a}}(\mathbf{x}(i))) + (1 - y(i)) \log(1 - h_{\mathbf{a}}(\mathbf{x}(i)))$$
- É a média aritmética do erro para cada amostra.
- Ela mede quão diferente está a distribuição prevista pelo modelo da distribuição real dos rótulos.
 - Assim, o modelo aprende a aproximar a distribuição real.

Função de erro

$$J_e(\mathbf{a}) = -\frac{1}{N} \sum_{i=0}^{N-1} \underbrace{y(i) \log(h_{\mathbf{a}}(\mathbf{x}(i)))}_{\text{Só exerce influência no erro se } y(i)=1} + \underbrace{(1 - y(i)) \log(1 - h_{\mathbf{a}}(\mathbf{x}(i)))}_{\text{Só exerce influência no erro se } y(i)=0} .$$

- Quando $y(i) = 1$
 - $-\log(h_{\mathbf{a}}(\mathbf{x}(i))) \rightarrow \infty$ quando $h_{\mathbf{a}}(\mathbf{x}(i)) \rightarrow 0$.
 - $-\log(h_{\mathbf{a}}(\mathbf{x}(i))) \rightarrow 0$ quando $h_{\mathbf{a}}(\mathbf{x}(i)) \rightarrow 1$.
- Quando $y(i) = 0$
 - $-\log(1 - h_{\mathbf{a}}(\mathbf{x}(i))) \rightarrow \infty$ quando $h_{\mathbf{a}}(\mathbf{x}(i)) \rightarrow 1$.
 - $-\log(1 - h_{\mathbf{a}}(\mathbf{x}(i))) \rightarrow 0$ quando $h_{\mathbf{a}}(\mathbf{x}(i)) \rightarrow 0$.
- O erro é muito alto se o classificador erra e tende a zero se acerta.

Função de erro

- Uma má notícia, **não existe uma equação analítica** que nos dá diretamente os **pesos** que minimizam essa **função de erro**.
 - Ou seja, não há um equivalente da **equação normal**.
- Entretanto, a boa notícia é que por ser **convexa** e **diferenciável**, **conseguimos** calcular o **vetor gradiente da função de erro** e **implementar** o algoritmo do **gradiente descendente (GD)**.
- Podemos implementar todas as versões do **GD**
 - Batelada
 - Estocástico
 - Mini-batch

Vetor gradiente

- A **atualização iterativa** dos **pesos** é dada por

$$\mathbf{a} = \mathbf{a} - \alpha \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}}.$$

- O **vetor gradiente** da **função da entropia cruzada** é dado por

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{1}{N} \sum_{i=0}^{N-1} \left[y(i) - \underbrace{h_a(\mathbf{x}(i))}_{\hat{\mathbf{y}}} \right] \mathbf{x}(i) = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}).$$

Forma matricial

onde $\mathbf{X} \in \mathbb{R}^{N \times K+1}$ é a matriz de atributos, $\mathbf{y}$ e $\hat{\mathbf{y}} \in \mathbb{R}^{N \times 1}$ são vetores contendo todos os valores esperados e preditos, respectivamente.

- O anexo I apresenta os passos para se encontrar o vetor gradiente.

Processo de treinamento

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{1}{N} \sum_{i=0}^{N-1} [y(i) - h_{\mathbf{a}}(\mathbf{x}(i))] \mathbf{x}(i) = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}})$$

- Para encontrar o **vetor gradiente**, a função $g(\mathbf{x})$ foi considerada como sendo um **hiperplano**, mas o resultado pode ser diretamente estendido para polinômios.
 - Cada coluna de $\mathbf{X}$ é formada pelas combinações dos atributos originais de cada monômio do polinômio.
- O **vetor gradiente** acima é **idêntico** (exceto pela constante 2) àquele obtido para a **regressão linear** utilizando a função de **erro quadrático médio**.

Pseudocódigo do gradiente descendente

- De posse do **vetor gradiente**, podemos usá-lo com qualquer versão do **gradiente descendente** para atualizar os pesos.

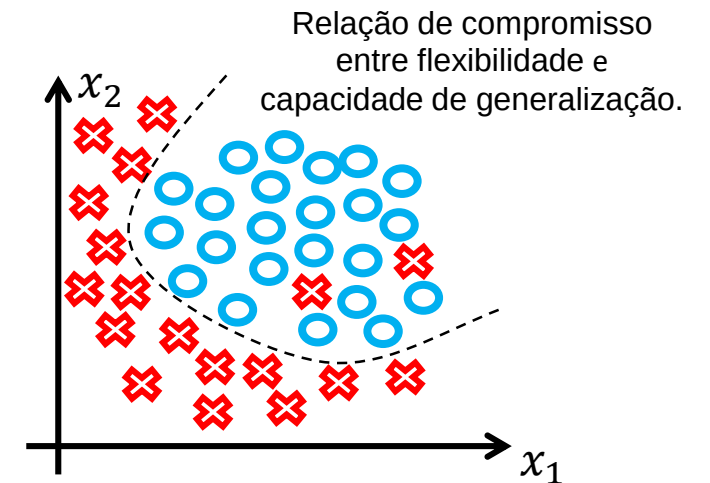
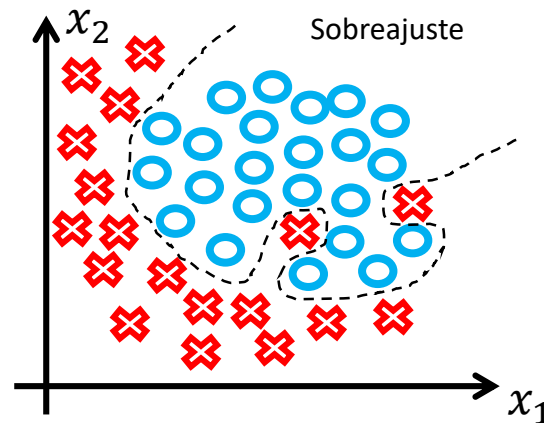
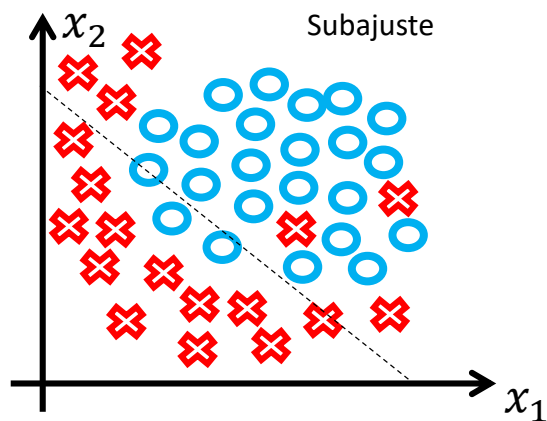
$\mathbf{a} \leftarrow$ inicializa em um ponto qualquer do espaço de pesos
loop até convergir ou atingir o número máximo de iterações do

$$\mathbf{a} \leftarrow \mathbf{a} - \alpha \frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} \text{ (eq. de atualização dos pesos)}$$

- Tudo o que aprendemos na regressão linear com relação ao treinamento de modelos com o gradiente descendente, incluindo os valores de α , se aplica ao treinamento de regressores logísticos.

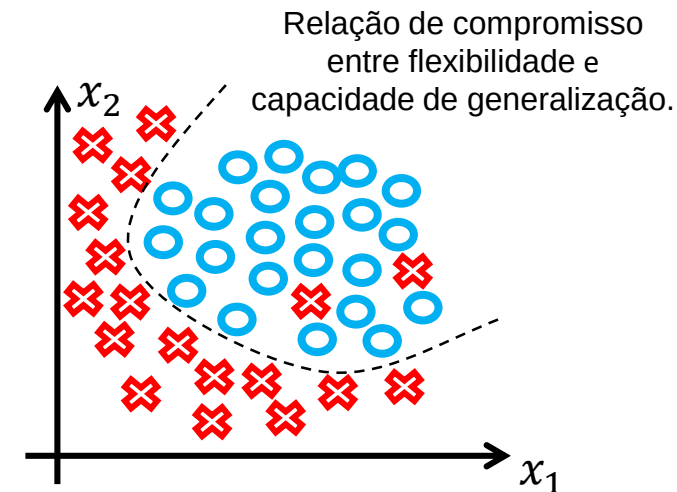
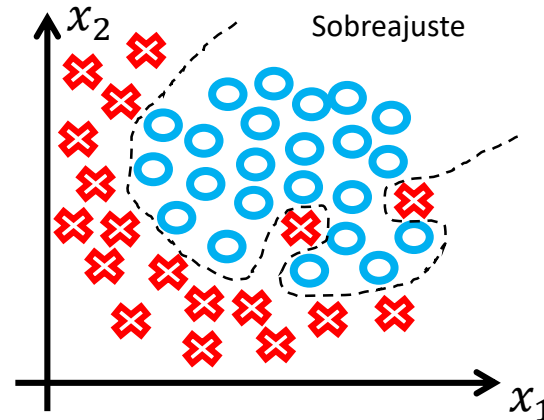
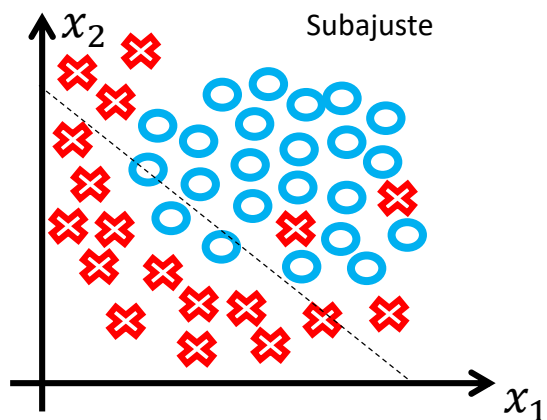
Subajuste e sobreajuste

- Assim como ocorre com a **regressão linear**, modelos de **regressão logística** também estão sujeitos à **sobreajuste** e **subajuste**.
- Qual deve ser a complexidade, no caso de polinômios, o grau, da **função discriminante**, $g(\mathbf{x})$?



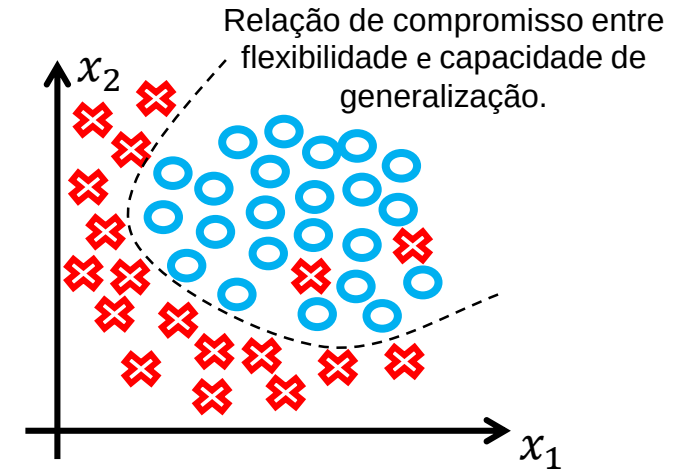
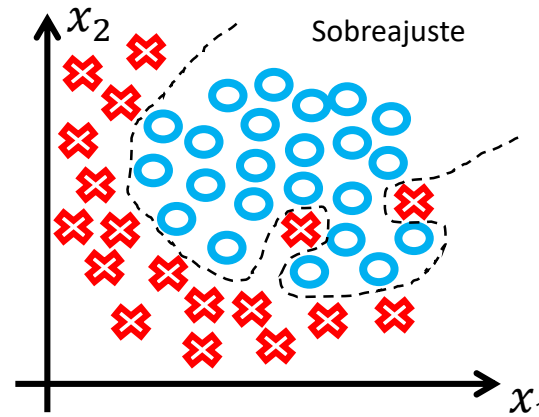
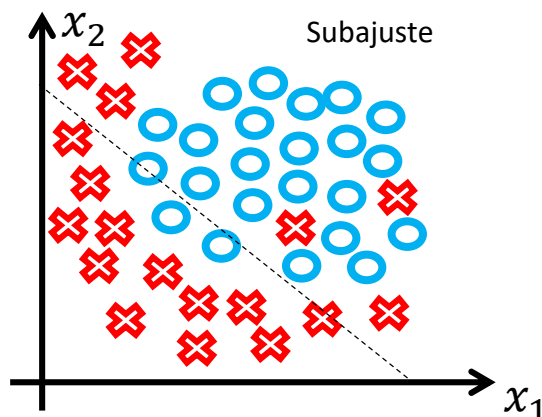
Subajuste e sobreajuste

- Na primeira figura, o polinômio (i.e., reta) tem uma **baixa flexibilidade**.
- Consequentemente, o modelo apresenta uma **baixa capacidade de generalização**.
- Assim, os erros nos conjuntos de treinamento e teste seriam **ambos altos**.
- Ou seja, o classificador erraria a classificação de boa parte dos exemplos.



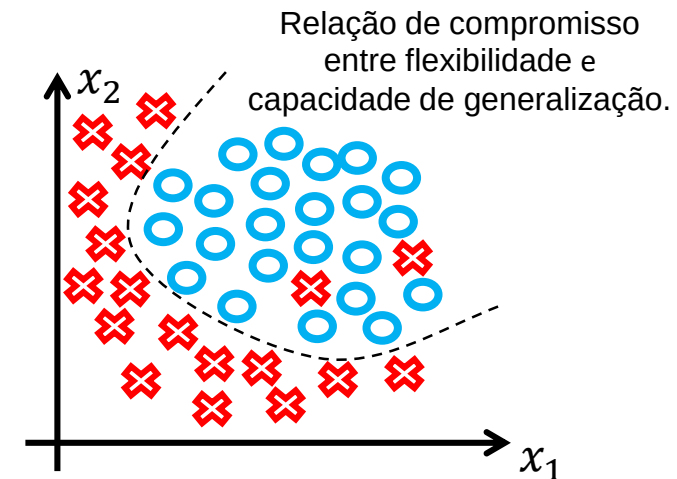
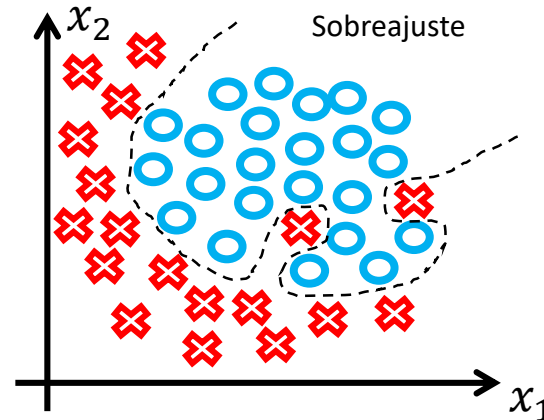
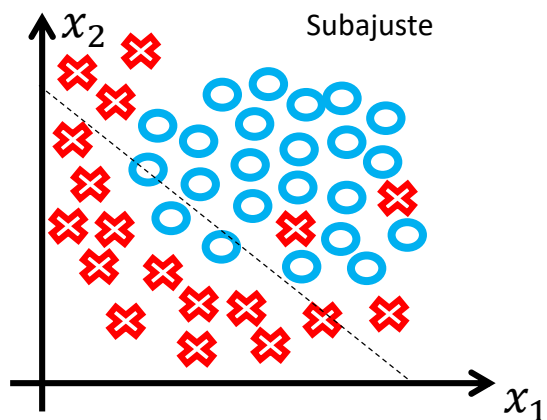
Subajuste e sobreajuste

- Na segunda figura, a **flexibilidade excessiva** do polinômio (i.e., grau elevado) dá origem a contorções na **fronteira de decisão** na tentativa de **minimizar o erro** de classificação **junto aos dados de treinamento**.
- Porém, o classificador cometerá muitos erros para dados inéditos, ou seja, **não irá generalizar bem**.
- Nesse caso o classificador apresenta **erro de treinamento muito baixo** e **erro de teste muito alto**.



Subajuste e sobreajuste

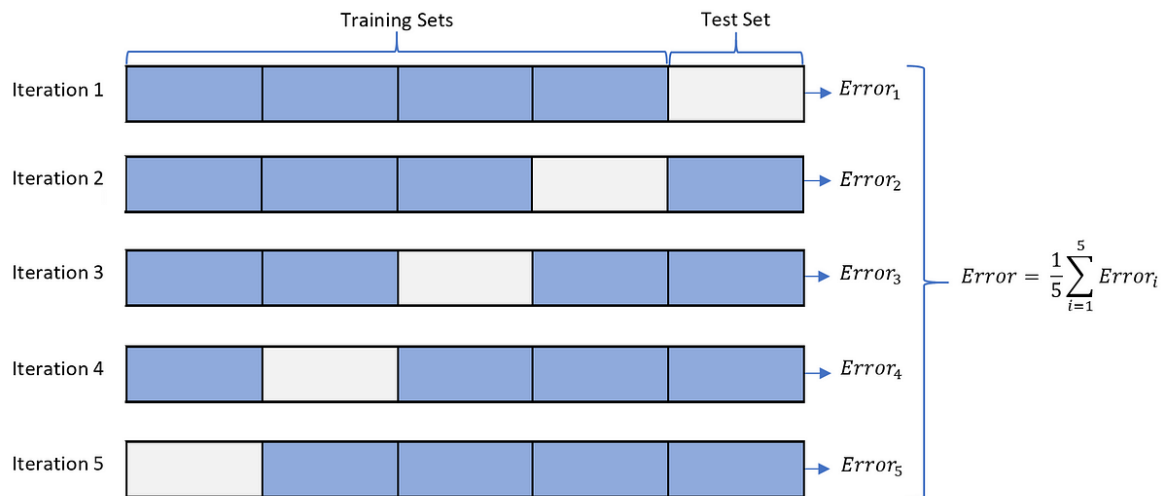
- Já a última figura mostra o que seria uma boa **hipótese de classificação**.
- O polinômio apresenta uma **boa relação de compromisso** entre a **flexibilidade** e a **capacidade de generalização**.
- Os erros nos conjuntos de treinamento e de teste seriam **baixos e próximos**.



*Como evitamos o subajuste e o
sobreajuste?*

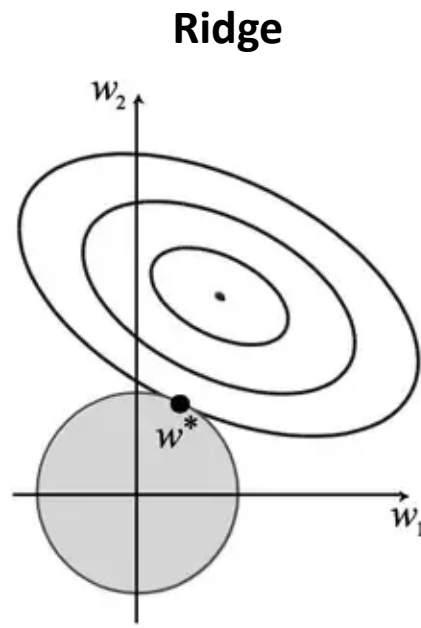
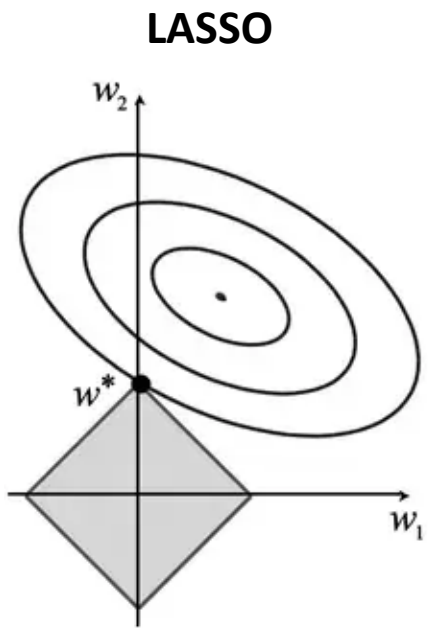
Como evitamos o subajuste e sobreajuste?

k-fold



- Podemos usar **técnicas de validação cruzada** como o *k-fold* para encontrar a complexidade ideal para o modelo.
- Essas técnicas nos auxiliam a encontrar um modelo que apresente uma **relação de compromisso entre flexibilidade e capacidade de generalização**, evitando assim os dois problemas.

Como evitamos o subajuste e sobreajuste?



- Uma forma de **evitar apenas problemas de sobreajuste** é usar **técnicas de regularização** (e.g., LASSO, Ridge, Elastic-Net, Early-stopping).
- A regularização **reduz a flexibilidade** do modelo por meio de **penalizações** aplicadas a seus pesos.
 - Modelos que se **sobreajustam** têm **pesos com valores absolutos muito altos**.
 - O que as técnicas de regularização fazem é **restringir o aumento** dos pesos a uma **região** de possíveis valores.

Exemplo

- [Exemplo: regressão logística.ipynb](#)



Lista de exercícios

Lista #6 - Classificação

Perguntas?

Obrigado!

Anexo I: Encontrando o vetor gradiente do regressor logístico

Encontrando o vetor gradiente

- Antes de encontrarmos o **vetor gradiente** de $J_e(\mathbf{a})$, vamos reescrever a **função de erro** utilizando as seguintes equivalências

$$\log(h_{\mathbf{a}}(\mathbf{x}(i))) = \log\left(\frac{1}{1+e^{-\mathbf{x}(i)^T \mathbf{a}}}\right) = -\log\left(1 + e^{-\mathbf{x}(i)^T \mathbf{a}}\right),$$

$$\log(1 - h_{\mathbf{a}}(\mathbf{x}(i))) = \log\left(1 - \frac{1}{1+e^{-\mathbf{x}(i)^T \mathbf{a}}}\right) = -\mathbf{x}(i)^T \mathbf{a} - \log\left(1 + e^{-\mathbf{x}(i)^T \mathbf{a}}\right).$$

- Assim, a nova expressão para a **função de erro médio** é dada por

$$\begin{aligned} J_e(\mathbf{a}) &= -\frac{1}{N} \sum_{i=0}^{N-1} -y(i) \log\left(1 + e^{-\mathbf{x}(i)^T \mathbf{a}}\right) + (1 \\ &\quad - y(i)) \left[-\mathbf{x}(i)^T \mathbf{a} - \log\left(1 + e^{-\mathbf{x}(i)^T \mathbf{a}}\right) \right] \end{aligned}$$

Encontrando o vetor gradiente

- O termo $-y(i) \log(1 + e^{-x(i)^T \mathbf{a}})$ é cancelado com um dos elementos gerados a partir do produto envolvido no segundo termo, de forma que

$$J_e(\mathbf{a}) = -\frac{1}{N} \sum_{i=0}^{N-1} -\mathbf{x}(i)^T \mathbf{a} + y(i) \mathbf{x}(i)^T \mathbf{a} - \log(1 + e^{-x(i)^T \mathbf{a}}).$$

- Se $-\mathbf{x}(i)^T \mathbf{a} = -\log(e^{x(i)^T \mathbf{a}})$, então

$$-\mathbf{x}(i)^T \mathbf{a} - \log(1 + e^{-x(i)^T \mathbf{a}}) = -\log(1 + e^{x(i)^T \mathbf{a}}).$$

- Desta forma, a **função de erro médio** se torna

$$J_e(\mathbf{a}) = -\frac{1}{N} \sum_{i=0}^{N-1} y(i) \mathbf{x}(i)^T \mathbf{a} - \log(1 + e^{x(i)^T \mathbf{a}}).$$

- Em seguida, encontramos o **vetor gradiente** de cada termo da equação acima.

Encontrando o vetor gradiente

- Assim, o **vetor gradiente** do primeiro termo da equação anterior é dado por

$$\frac{\partial [y(i)\mathbf{x}(i)^T \mathbf{a}]}{\partial \mathbf{a}} = y(i)\mathbf{x}(i)$$

- O **vetor gradiente** do segundo termo da equação anterior é dado por

$$\begin{aligned} \frac{\partial \left[\log \left(1 + e^{\mathbf{x}(i)^T \mathbf{a}} \right) \right]}{\partial \mathbf{a}} &= \frac{1}{1 + e^{\mathbf{x}(i)^T \mathbf{a}}} e^{\mathbf{x}(i)^T \mathbf{a}} \mathbf{x}(i) \\ &= \frac{1}{1 + e^{-\mathbf{x}(i)^T \mathbf{a}}} \mathbf{x}(i) \\ &= h_{\mathbf{a}}(\mathbf{x}(i)) \mathbf{x}(i). \end{aligned}$$


- Usamos a **regra da cadeia** para encontrar o vetor gradiente do segundo termo.

Encontrando o vetor gradiente

- Portanto, combinando os 2 resultados anteriores, temos que o **vetor gradiente** da **função de erro médio** é dado por

$$\frac{\partial J_e(\mathbf{a})}{\partial \mathbf{a}} = -\frac{1}{N} \sum_{i=0}^{N-1} [y(i) - h_a(\mathbf{x}(i))] \mathbf{x}(i) = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}).$$

Forma matricial:
 $\mathbf{X} \in \mathbb{R}^{N \times K+1}$, $\mathbf{y}$
e $\hat{\mathbf{y}} \in \mathbb{R}^{N \times 1}$

